Chapter 5: Loops and Iteration (Repeated Steps)

1. Indefinite Loops (while)

A while loop tells Python to repeatedly execute a block of code **as long as a certain condition remains True**. It is called "indefinite" because it runs until the condition becomes False, however long that takes.

The Basic Structure:

```
n = 5
while n > 0:
    print(n)
    n = n - 1
print("Done!")
```

- **Explanation:** Executes the block while the condition is True. The output is 5 4 3 2 1 Done!
- ⚠️ **The Danger (Infinite Loops):** Loops with conditions that never become False will run forever. You *must* have an **iteration variable** (like n above) that changes each time through the loop to eventually make the condition False.

2. Loop Control: break and continue

Keyword	Function	Example Concept
break	Ends the current loop immediately and jumps out.	The "Emergency Exit."
continue	Skips the rest of the current iteration and jumps to the next iteration.	The "Skip to Next" button.

# Example with break	# Example with continue

`while True:`
`  line = input('> ')`
`  if line == 'done':`
`  break #Exits the loop`
`  print(line)`
 	
`print('Done!')`
	`while True:`
	`  line = input('> ')`
	`  if line.startswith('#'):`
	`  continue # Skips printing`
	`  if line == 'done':`
	`  break`
	`  print(line)`
	
	`print('Done!')`

3. Definite Loops (for)

While while loops run until a condition is met, for loops run for a predetermined number of times. They automatically iterate through a specific **sequence** or set of items.

The Basic Structure:

```
numbers = [1, 2, 3, 4, 5]
for i in numbers:
      print(i)
print("Done!")
```

Common Loop Idioms (Patterns):

Counting Items	Summing Items

`count = 0`
`for itervar in [3, 41, 12, 9, 74, 15]:`
`  count = count + 1`
 	
`print('Count:', count)`
	`total = 0`
	`for itervar in [3, 41, 12, 9, 74, 15]:`
	`  total = total + itervar`
	
	`print('Total:', total)`

4. Advanced Loop Idioms & The is Operator

To find the largest or smallest value, we use a loop to scan through data. We use None as an empty baseline to start.

Teacher's Note: The is and is not operators are used for identity checks. They are stronger than == because they check if two variables are the *exact same object in memory*, not just if they have the same value.

Finding Minimum and Maximum:

Finding the largest value	Finding the smallest value

`largest = None`
`for itervar in [3, 41, 12, 9, 74, 15]:`
`  if largest is None or`
`  itervar > largest:`
 	
`  largest = itervar`
 	
`print('Largest:', largest)`
	`smallest = None`
	`for itervar in [3, 41, 12, 9, 74, 15]:`
	`  if smallest is None or`
	`  itervar < smallest:`
	
	`  smallest = itervar`
	
	`print('Smallest:', smallest)`